

Assignment 1 - Part B

B.1 Big-O Identification

Segment 1

Big-O: $O(1)$

The time complexity is $O(1)$. The program only accesses the first element of the array and prints it. The number of operations stays the same no matter how large the array is.

Segment 2

Big-O: $O(n)$

The time complexity is $O(n)$. The loop goes through every element in the array one time. As n gets larger, the number of operations increases at about the same rate.

Segment 3

Big-O: $O(\log n)$

The time complexity is $O(\log n)$. Each time the loop runs, the value of i is divided by 2. Because the value keeps getting cut in half, the number of loop iterations grows slowly as n gets larger.

Segment 4

Big-O: $O(n * m)$

The time complexity is $O(n * m)$. The outer loop runs n times, and the inner loop runs m times for every outer loop iteration. This means the total amount of work is based on n multiplied by m .

Segment 5

Big-O: $O(n + m)$

The time complexity is $O(n + m)$. The first loop runs n times and the second loop runs m times. Since the loops are separate and not nested, their running times are added together.

B.2 Empirical vs. Theoretical

Prediction

I predict that the `calculateAverage` method has a time complexity of $O(n)$. The method uses one loop that goes through every element in the `grades` array one time. As the size of the array increases, the amount of work should increase at about the same rate.

Timing Results

Small array size: 1,000 elements
Runtime: 8,400 nanoseconds

Large array size: 100,000 elements
Runtime: 788,600 nanoseconds

Comparison

The larger array took much longer to process than the smaller array. This matches my prediction of $O(n)$ because the method has to go through every element in the array. The runtime did not increase by the exact same amount as the array size, which can happen because of Java runtime behavior and other processes running on the computer. Overall, the results support the $O(n)$ time complexity.